Lecture 2: Layering, Framing & Error Handling

ECS 152A: Computer Networks

Alexander Gamero-Garrido

Our Problem

- Communications is more complicated than it seems
  - Modulation bits into analog signals
  - Splitting sequences of bits into packets
  - Fixing errors
  - Controlling access to network
  - Routing
  - Recovering from lost messages
  - Etc.…
- Really hard to think about all of this and get it right
- Not all applications need all of it
- How to achieve interoperability?

Layering: A Modular Approach

- Sub-divide the problem
  - Each layer relies on services from layer below
  - Each layer exports services to layer above
- Interface between layers defines interaction
  - Hides implementation details (encapsulation)
  - Layers can change without disturbing other layers (modularity)
- Interface among peers in a layer is a protocol
  - If peers speak same protocol, they can interoperate

Key Design Decision…

- …how do you divide functionality across the layers?

- **End-to-end argument [Saltzer84]**
  - Functionality **should** be implemented at a **lower layer** iff it can be **correctly** and **completely** implemented there
  - Incomplete versions of a function can be used as a performance enhancement, but not for correctness
- Early, and still relevant, example
  - ARPAnet (link layer) provided:
    reliable transfers across links between switches
  - Was this enough for reliable communication?
  - No, packets could still get corrupted on host-switch link, or inside of the switches
  - Hence, **still need reliability** at higher layers

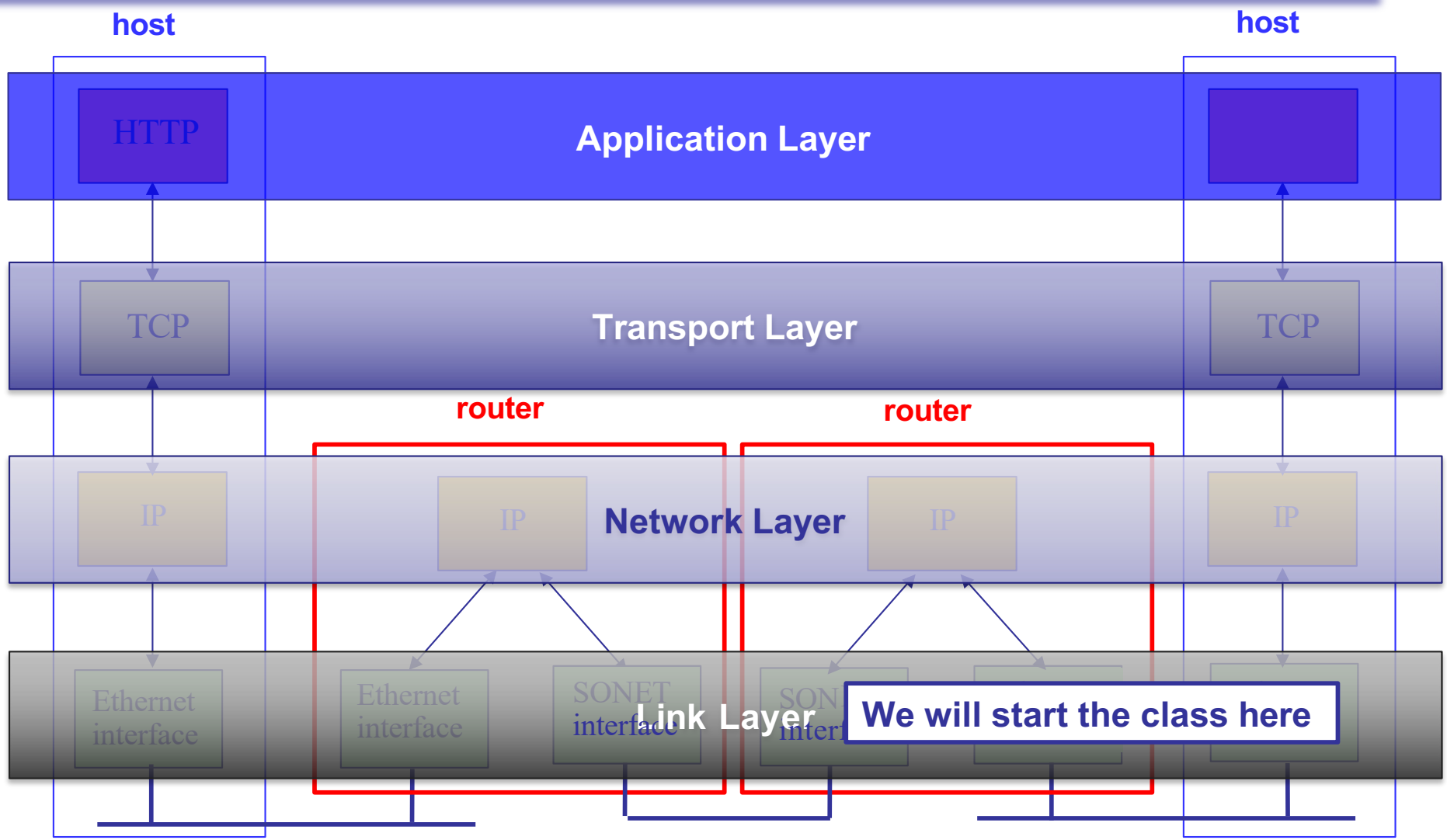
Protocol Standardization

- Communicating hosts need to speak the same protocol
  - Standardization to enable multiple implementations
  - Or, the same folks have to write all the software

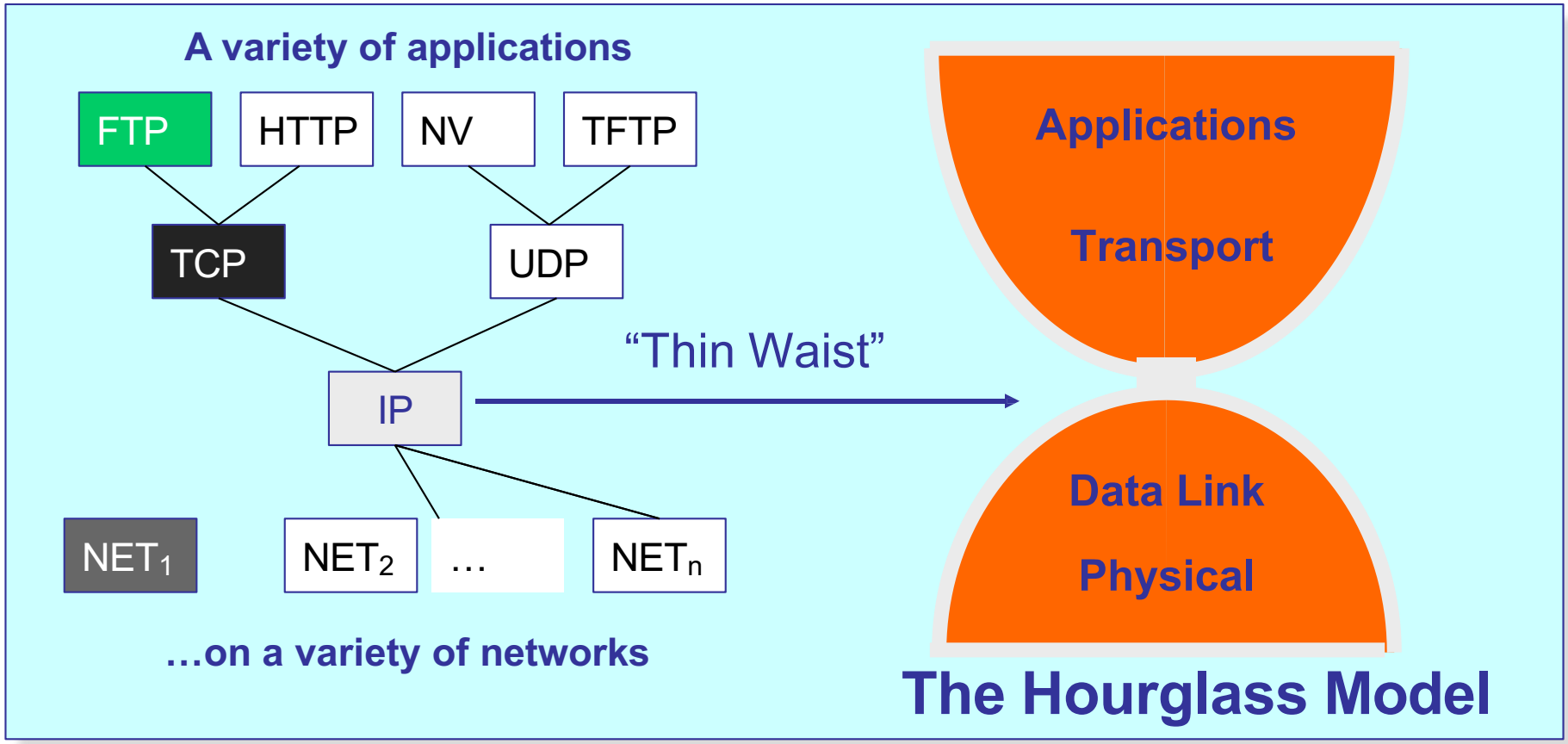
- Internet Engineering Task Force
  - Based on working groups that focus on specific issues
  - Produces “Request For Comments” (RFCs)
    - » “Rough consensus and running code”
    - » After enough time passes, promoted to Internet Standards

- Other standards bodies exist
  - ISO, ITU, IEEE, etc.

TCP/IP Protocol Stack



Layering enabled growth: Internet Protocol Suite



The success of the Internet (and its problems) can largely be attributed to the "thin waist"

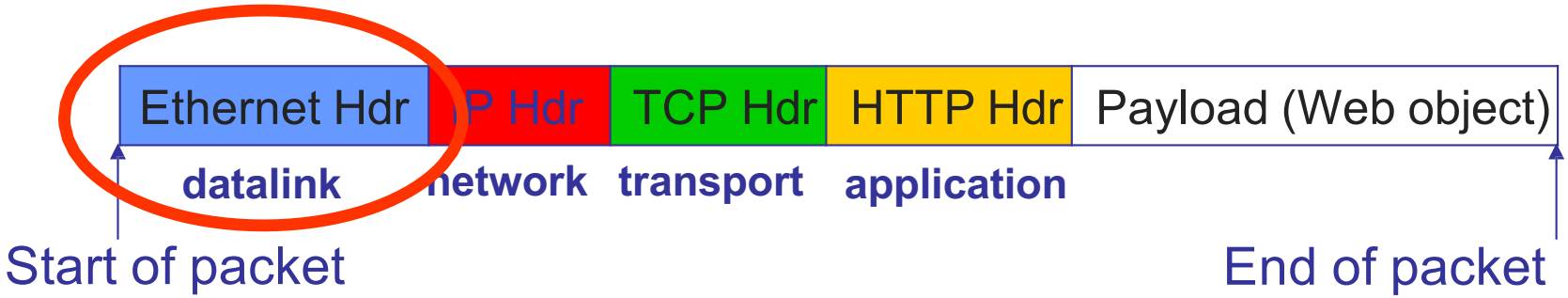
Activity 1: Narrow waist

Discuss in pairs for ~2 minutes:

What are some potential advantages and disadvantages of having a single narrow waist on the Internet?

Layer context encapsulated in packets via headers

- Typical packet in Web transfer



- Notice that layers (and thus headers) add overhead
  - Space (headers), effective bandwidth
  - Time (processing headers, “peeling the onion”), latency

Phy/(MAC)Link (Layer 1)

- Signal encoding
  - Encode binary data from source node into signals that physical links carry
  - Signal is decoded back into binary data at receiving node
  - Work performed by network adapter at sender and receiver
- Media access
  - Arbitrate which nodes can send frames at any point in time
  - Not always necessary; e.g. point-to-point duplex links

The first layer we will discuss: (Data) Link (Layer 2)

- **Framing**
  - Break stream of bits up into discrete chunks
  - **Multiplexing**
    - » Share a link with multiple sources and destinations
    - » Also not always required; again, point-to-point

- **Error handling**
  - Detect and/or correct errors in received frames

Framing

Break down a stream of bits into smaller, digestible chunks called frames

- Identifies the beginning and end of a chunk of information
- Allows the link to be shared (**Multiplexing**)
  - Multiple senders and/or receivers can **time multiplex** the link
    - » One frame sent at a time
  - Each frame can be separately addressed (different src/dest)
- Provides manageable unit for **error handling**
  - Easy to determine whether something went wrong in a chunk of data
  - And perhaps even to fix it if desired

What's in a Frame?

- Wraps payload bits up with some additional information
  - Header usually contains addressing information
  - Maybe includes a trailer (w/checksum or CRC—next lecture)
- Basic unit of communication
  - Link either delivers entire frame payload, or none of it
  - Typically some maximum transmission unit (MTU)
- Some link layers require absence of frames as well
  - I.e., minimum gaps between frames

Where is the beginning/end?

0100100100100000011101000110100001101001011011100110101100100000011101000110100001100001011101000010000001001 0
0100100000011100110110100001100001011011000110110000100000011011100110010101110110011001010111001000100000011 1
0011011001010110010100010100100000100100000011001110111001001100001011100000110100000100000011011010110111101 1
1001001100101001000000110110001101111011101100110010101101100011110010010000001110100011010000110000101101110 0
0100000011000010010000001110100011100100110010101100101001011100001010010000010010000001110100011100100110010 1
0110010100100000011101110110100001101111011100110110010100100000011000110111001001110101011000110110100101100 0
0101101100001000000111000001110010011011110111000001100101011100100111010001111001000101001001001011100110010 0
0000110110001101111011011110111000000101101011001100111001001100101011001010010000001100011011011110110111001 1
0111001100101011000110111010001101001011101100110100101110100011110010010111000010100100000100100000011101000 1
1100100110010101100101001000000111010001101000011000010111010000100000011011010111010101110011011110100001000 0
0110001001100101001000000111001101110101011100100110010100100000011101000110111100100000011100110111000001100 0
0101101110000101001010011011011110010000001110000011000010110001101101011011001010111010001110011001000000110 0
0110110000101101110001000000111001001100101011000010110001101101000010000001100101011101100110010101110010011 1
1100100100000010011000100000101001110001011100001010010001100110100101110010011100110111010000101100001000000 1
1101000110100001100101001000000111001001101111011011110111010000100000011011010111010101110011011110100001000 0
0110001001100101001000000111001101100101011011000110010101100011011110100011001010110010000101110000101001000 1
0011110010010000001001001010001000010110000100000011010010111010000100000011010010111001100100000001100101011 1
1000110010101100011011110100011001010110010000101110000101001001100011001010110000101110011011110100001011010 1
0011011011110111001101110100001000000111000001100001011101000110100001110011001000000011001100111001001101111 1
1011010010000001110010011011110110111101110100001000000110000101110010011001010010000001110100011100100110000 1
0110001101100101011001000010111000010100100100101101110001000000111010001101000011001010010000001110100011100 1
0011001010110010100101100001000000111010001101000011001010111001101100101001000000111000001100001011101000110 1
0000111001100100000011000010111001001100101001000000111000001101100011000010110001101100101011001000010111000 0
1010010000010010000001101101011001010111001101101000001000000110100101110011001000000011011010110000101100100 1
1001010010000000110001001111001001000000110011001101111011011000110101101110011001000000011011000110100101101 1
0110010100100000011011010110010100101100000010100101010001101000011001010110111000100000011000100111001001101 0
1011001000110011101100101011100110010000001100110011010010110111001100100001000000110000100100000011100110111 0
0000110000101101110011011100110100101101110011001110010000001110100011100100110010101100101001011 0

Activity 2: discuss in pairs where each frame should start/end

0100100100100000011101000110100001101001011011100110101100100000011101000110100001100001011101000010000001001 0
0100100000011100110110100001100001011011000110110000100000011011100110010101110110011001010111001000100000011 1
0011011001010110010100010100100000100100000011001110111001001100001011100000110100000100000011011010110111101 1
1001001100101001000000110110001101111011101100110010101101100011110010010000001110100011010000110000101101110 0
0100000011000010010000001110100011100100110010101100101001011100001010010000010010000001110100011100100110010 1
0110010100100000011101110110100001101111011100110110010100100000011000110111001001110101011000110110100101100 0
0101101100001000000111000001110010011011110111000001100101011100100111010001111001000101001001001011100110010 0
0000110110001101111011011110111000000101101011001100111001001100101011001010010000001100011011011110110111001 1
0111001100101011000110111010001101001011101100110100101110100011110010010111000010100100000100100000011101000 1
1100100110010101100101001000000111010001101000011000010111010000100000011011010111010101110011011110100001000 0
0110001001100101001000000111001101110101011100100110010100100000011101000110111100100000011100110111000001100 0
0101101110000101001010011011011110010000001110000011000010110001101101011011001010111010001110011001000000110 0
0110110000101101110001000000111001001100101011000010110001101101000010000001100101011101100110010101110010011 1
1100100100000010011000100000101001110001011100001010010001100110100101110010011100110111010000101100001000000 1
1101000110100001100101001000000111001001101111011011110111010000100000011011010111010101110011011110100001000 0
0110001001100101001000000111001101100101011011000110010101100011011110100011001010110010000101110000101001000 1
0011110010010000001001001010001000010110000100000011010010111010000100000011010010111001100100000001100101011 01
1000110010101100011011110100011001010110010000101110000101001001100011001010110000101110011011110100001011010110
0011011011110111001101110100001000000111000001100001011101000110100001110011001000000011001100111001001101111 01
1011010010000001110010011011110110111101110100001000000110000101110010011001010010000001110100011100100110000 1
0110001101100101011001000010111000010100100100101101110001000000111010001101000011001010010000001110100011100 1
0011001010110010100101100001000000111010001101000011001010111001101100101001000000111000001100001011101000110 1
0000111001100100000011000010111001001100101001000000111000001101100011000010110001101100101011001000010111000 0
1010010000010010000001101101011001010111001101101000001000000110100101110011001000000011011010110000101100100 01
1001010010000001100010011110010010000001100110011011110110110001101011011100110010000001101100011010010110101 1
0110010100100000011011010110010100101100000101001010100011010000110010101101110001000000110001001110010011010 0
1011001000110011101100101011100110010000001100110011010010110111001100100001000000110000100100000011100110111 0
0000110000101101110011011100110100101101110011001110010000001110100011100100110010101100101001011 0

Identifying Frames

- First task is to **delineate** frames
  - Receiver needs to know when a frame **starts** and **ends**
  - Otherwise, errors from misinterpretation of data stream

- Several different alternatives
  - 1. Fixed length (bits) frames
  - Explicitly delimited frames
    - » 2. Length-based framing
    - » 3. Sentinel-based framing
  - 4. Fixed duration (seconds) frames

#1 Fixed-Length Frames

- Easy to manage for receiver
  - Well understood buffering requirements
- Introduces inefficiencies for variable length payloads
  - Larger payloads need to be fragmented across many frames
  - May waste bandwidth (what waste?)
  - Very common implementation inside switches
- Requires explicit design tradeoff
  - ATM uses 53-byte frames (cells)
  - Why 53? 48 + 5. Why 48?

#2 Length-Based Framing

- To avoid overhead, we'd like variable length frames
  - Each frame declares how long it is
  - E.g. DECNet DDCMP
- What's the issue with explicit length field?
  - Must correctly read the length field (bad if corrupted)
    - » Need to decode *while* receiving
  - Still need to identify the beginning…

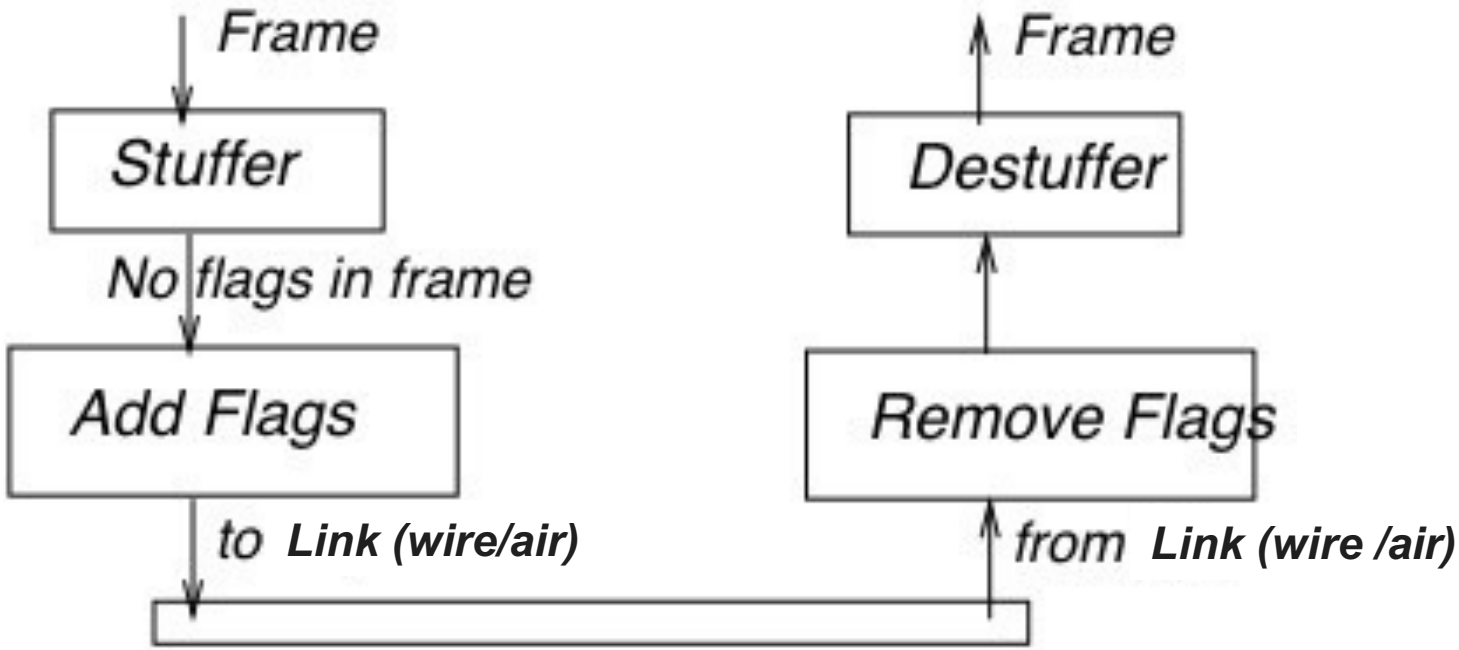
#3 Sentinel-based Framing

- Allow for variable length frames
- Idea: mark start/end of frame with special "marker"
  - Byte pattern, bit pattern, signal pattern
- But… must make sure **marker doesn't appear in data**

- Two solutions
  - Special non-data symbol used to convey sentinel
    - » Impact on efficiency (can't use symbol for data) of code
  - Stuffing
    - » Dynamically remove marker bit patterns from data stream
    - » Receiver "unstuffs" data stream to reconstruct original data

Stuffing

- Insert bytes/bits into data stream to make sure that sentinel (flag) does not appear in payload



Bit-level Stuffing

- Avoid sentinel flag bit pattern in payload data
  - Commonly, sentinel flag is bit pattern 01111110 (0x7E)
  - Invented for SDLC/HDLC (IBM layer 2), now standard pattern
- Sender: any time five ones appear in outgoing data, insert a zero, resulting in 0111110

Stuffed bits

0111111000011101110111111011111001

0111110100001110111011111001111100 01

- Receiver: any time five ones appear, removes next zero
  - If there is no zero, there will either be six 1s and a 0 (sentinel) or
  - It declares an error condition!
  - Note bit pattern that cannot appear is 01111111 (0x7F)

For Next Class

- Read 2.5, 6.1-6.2 in P&D